

Nota metodologica

Valutare modelli ML nel data plane

Come lo fa la letteratura,
come siamo messi, cosa manca

Progetto **IPA/eBPF** — pipeline hardcoded, template, modular
topologia Germany50 su Kathara

Settembre 2026

Indice

1	Perché questa nota	2
2	Che cosa misura la letteratura	3
2.1	Asse 1 — Qualità del modello	3
2.2	Asse 2 — Costo per pacchetto	3
2.3	Asse 3 — Costo in risorse	4
2.4	Asse 4 — Costo di aggiornamento e flessibilità	4
3	Con quali strumenti	5
3.1	Il protocollo standard: RFC 2544	5
3.2	I generatori di traffico	5
3.3	Il microbenchmark kernel-side: BPF_PROG_TEST_RUN	5
4	Le trappole documentate	7
4.1	Il bias di misura è la norma, non l’eccezione	7
4.2	Un microbenchmark più veloce non è un sistema più veloce	7
4.3	Ambienti astratti non predicono ambienti reali	8
5	Dove siamo	9
5.1	Quadro sinottico	9
5.2	Dove siamo forti	10
5.3	Dove siamo scoperti	10
5.3.1	Non sappiamo se il modello è buono	10
5.3.2	Il throughput riportato non è un throughput	10
5.3.3	Due feature del modello sono inerti	10
5.3.4	Una sola macchina, una sola sessione	11
6	Che cosa fare	12
6.1	Priorità 1 — Misurare la qualità del modello	12
6.2	Priorità 2 — Rendere onesto il throughput	12
6.3	Priorità 3 — Chiudere il ciclo end-to-end	13
6.4	Priorità 4 — Irrobustire le misure di latenza	13
6.5	Priorità 5 — Riparare o dichiarare le feature inerti	13
7	In sintesi	14

Perché questa nota

Il professore ha posto una domanda precisa: *come vengono effettuati i test per questi tipi di modelli in letteratura?* La domanda non è retorica. Il progetto misura già molte cose — istruzioni eBPF, codice jited, tail call, letture di mappa, memoria, latenza — e le misura con cura. Il punto è capire se l'insieme di quelle misure costituisce una *valutazione* nel senso in cui la intende la letteratura sul machine learning nel data plane, oppure se copre solo una parte del problema.

La risposta breve, anticipata qui e argomentata nel resto del documento, è che il progetto misura molto bene **una** delle quattro dimensioni che la letteratura considera obbligatorie, e non misura affatto le altre tre. Non è un difetto di esecuzione: le misure esistenti sono più rigorose della media. È un difetto di *copertura*.

Questa nota è organizzata così: il Capitolo 2 ricostruisce quali metriche la letteratura riporta e perché; il Capitolo 3 descrive con quali strumenti e protocolli le ottiene; il Capitolo 4 raccoglie gli errori metodologici documentati, che sono il motivo per cui certe scelte vanno giustificate esplicitamente; il Capitolo 5 confronta lo stato del progetto con quel quadro; il Capitolo 6 propone un piano di lavoro ordinato per priorità, con criteri di accettazione.

Tutti i riferimenti citati sono stati verificati: titolo, autori, sede e anno provengono dalle pagine ufficiali degli editori o dagli archivi degli autori, non dalla memoria. Dove un dato numerico proviene dal codice di questo progetto è indicato esplicitamente.

Che cosa misura la letteratura

Il campo ha un nome consolidato: *in-network machine learning*. Due rassegne recenti ne definiscono i confini e ne sistematizzano i risultati: quella di Zheng et al. su IEEE Communications Surveys & Tutorials [1] e quella di Parizotto et al. su ACM Computing Surveys [2]. Leggendole insieme ai lavori sperimentali principali emerge una struttura ricorrente: le valutazioni si articolano su quattro assi, e i lavori pubblicati li coprono tutti e quattro.

2.1 Asse 1 — Qualità del modello

È la prima cosa che ogni lavoro riporta, e in genere occupa la prima tabella. Non si tratta di prestazioni ma di *correttezza statistica*: quanto bene il modello svolge il compito per cui è stato addestrato. Le metriche sono quelle standard del machine learning — accuratezza, precision, recall, F1 — valutate su un dataset di test tenuto separato dall’addestramento.

Il punto cruciale, e quello più spesso trascurato da chi arriva al problema dal lato sistemistico, è che questa misura va riportata **due volte**: una per il modello originale in virgola mobile e una per il modello effettivamente eseguito nel dispositivo di rete, quantizzato e vincolato. La differenza fra le due è un risultato in sé, perché quantifica quanto si è pagato in qualità per poter eseguire il modello nel data plane.

N3IC [3] — già citato nella bibliografia della tesi — è esemplare in questo: compila reti neurali *binarie* nel data plane di SmartNIC, e siccome la binarizzazione è una forma estrema di quantizzazione, il lavoro è costretto a dimostrare che l’accuratezza resta accettabile su due casi d’uso reali (identificazione del traffico e anomaly detection). IIsy [4] va oltre e fa della perdita di qualità un elemento di progetto: adotta un modello “piccolo” sullo switch e uno “grande” nel backend, e usa la confidenza della decisione per stabilire quando il modello in rete basta e quando serve inoltrare a valle.

Che cosa comporta per noi

Il progetto non misura questo asse in nessuna forma. L’accuratezza è trattata come *invariante* fra le tre pipeline, il che è metodologicamente corretto per confrontarle fra loro, ma non dice nulla su quanto il modello sia buono nel suo compito, né quanto costi la quantizzazione a 8 bit.

2.2 Asse 2 — Costo per pacchetto

Latenza e throughput. È l’asse su cui il progetto è già forte, ed è anche quello dove la letteratura è più esigente sul *come* si misura — questione rinviata al Capitolo 3.

Due precisazioni sulle unità. Primo: la latenza riportata nei lavori seri è quella del dispositivo sotto test, isolata dal resto, e accompagnata da un *riferimento* — tipicamente lo stesso percorso senza inferenza, per separare il costo del modello dal costo dell’infrastruttura di forwarding. Secondo: il throughput è una grandezza *misurata*, non derivata dalla latenza. Sono due cose diverse, e il Capitolo 4 spiega perché confonderle è un problema.

I numeri tipici danno la scala del campo: N3IC riporta latenza di classificazione fino a 100 volte inferiore e throughput da 1,5 a 7 volte superiore rispetto a sistemi software, mantenendo il forwarding a line rate su NIC a 40 Gbps [3].

2.3 Asse 3 — Costo in risorse

Quanto del dispositivo consuma il modello. La forma concreta dipende dal bersaglio: su switch programmabili si contano stadi della pipeline, tabelle match-action, memoria SRAM/TCAM; su FPGA si contano risorse logiche e blocchi di memoria — N3IC riporta meno dell'1–2% delle risorse di un Virtex7 [3]; su eBPF/XDP si contano istruzioni verificate, dimensione del codice JIT-ato e memoria delle mappe.

Questo asse ha un ruolo particolare: è l'unico completamente *deterministico*. Non dipende dal carico, dal rumore o dalla macchina, il che lo rende l'unica famiglia di metriche confrontabile fra esperimenti eseguiti in momenti e su hardware diversi.

2.4 Asse 4 — Costo di aggiornamento e flessibilità

Quanto costa cambiare il modello a sistema in esercizio, e che cosa si può cambiare senza ricompilare. È l'asse più recente e quello meno standardizzato, ma è centrale nei lavori che si presentano come *framework* anziché come singola implementazione: Planter [5] nasce esattamente per rendere rapido il ciclo modello-bersaglio-misura, e la sua tesi è che la velocità di prototipazione sia essa stessa un risultato.

Che cosa comporta per noi

Questo è l'asse su cui il progetto ha il contributo più originale. Il confronto fra le tre pipeline è una misura su questo asse, ed è già quantificata: ricompilazione contro scrittura di una mappa, con circa 926 ms misurati per la hardcoded contro frazioni di millisecondo per le altre due. Va valorizzato, non dato per scontato.

Con quali strumenti

3.1 Il protocollo standard: RFC 2544

Per il costo per pacchetto esiste uno standard, ed è vecchio e consolidato: la RFC 2544 [7], metodologia di benchmarking per dispositivi di interconnessione di rete. È la base su cui sono costruiti praticamente tutti gli strumenti commerciali di test.

Definisce poche cose, in modo stretto:

- **Throughput** — non “quanti pacchetti al secondo riesco a spingere”, ma il *massimo frame rate senza perdita di frame*. Si determina per ricerca binaria: si invia a un certo tasso, si contano i frame persi, si aggiusta.
- **Latency** — misurata secondo le definizioni della RFC 1242, distinguendo dispositivi store-and-forward da bit-forwarding.
- **Frame loss rate** — la percentuale di perdita a tassi di ingresso crescenti.
- **Back-to-back frames** — il comportamento in presenza di raffiche.

Due prescrizioni della RFC meritano attenzione perché toccano direttamente il nostro caso. La prima: ogni prova dovrebbe durare **almeno 120 secondi**. La seconda: la prova va ripetuta **almeno 20 volte** e si riporta la media dei valori registrati [7]. Sono requisiti pensati per hardware di rete, e non vanno trasportati alla cieca su un microbenchmark kernel-side, ma danno la misura di quanto la comunità consideri insufficiente una singola prova breve.

Tutte le grandezze vanno inoltre riportate per *diverse dimensioni di frame* — tipicamente da 64 byte in su — perché il costo per pacchetto e il costo per byte si comportano in modo opposto.

3.2 I generatori di traffico

La conseguenza pratica della RFC 2544 è che serve un generatore capace di saturare il collegamento e di contare con precisione. Il lavoro di riferimento su XDP [6] — che è la base tecnologica di questo progetto — usa il generatore **TRex** su schede Mellanox ConnectX-5 a 100Gbps, con CPU Xeon E5-1650 v4. Gli altri nomi ricorrenti nella letteratura sono MoonGen e pktgen-DPDK.

Il punto non è lo strumento specifico, ma il fatto che la misura del throughput richiede *traffico vero contro il dispositivo vero*. Non esiste in letteratura una misura di throughput accettata che sia stata derivata da un microbenchmark.

3.3 Il microbenchmark kernel-side: BPF_PROG_TEST_RUN

Questo progetto usa BPF_PROG_TEST_RUN, la primitiva che esegue un programma eBPF già caricato su un pacchetto sintetico direttamente nel kernel, restituendo il valore di ritorno e una misura di durata. È una scelta legittima e ben motivata nella tesi: aggira il problema della consegna end-to-end nell’emulazione.

Va però documentata per quello che è, perché ha limiti precisi. La documentazione del kernel è esplicita: in questa modalità i programmi **non hanno effetti collaterali**; in

particolare i pacchetti non vengono realmente rediretti né scartati, e il valore di ritorno viene semplicemente riportato allo spazio utente [11].

Questo significa che sotto `BPF_PROG_TEST_RUN`:

- il costo di `bpf_redirect` non viene pagato — non si attraversa driver né coda di trasmissione;
- non esiste una NIC, quindi niente DMA, niente interrupt, niente pressione sulla cache dovuta al traffico reale;
- non c'è perdita di pacchetti, quindi il throughput *a perdita nulla* della RFC 2544 non è nemmeno definibile.

È un ottimo strumento per confrontare varianti dello stesso programma in condizioni identiche — che è esattamente l'uso che ne fa la Tabella del confronto fra pipeline. Non è uno strumento per affermare quanto il sistema regga in esercizio.

Le trappole documentate

Questo capitolo raccoglie tre risultati metodologici pubblicati. Non sono opinioni: sono lavori che hanno misurato l'errore che si commette seguendo l'intuizione.

4.1 Il bias di misura è la norma, non l'eccezione

Mytkowicz et al. [8] hanno mostrato che modificare un aspetto apparentemente innocuo del setup sperimentale basta a portare un ricercatore a conclusioni sbagliate. Nei loro esperimenti il bias di misura si manifesta su tutte le architetture provate, con entrambi i compilatori provati, e sulla maggior parte dei programmi C di SPEC CPU2006. Gli effetti misurati sono sufficienti a sovrastimare un guadagno o addirittura a invertirne il segno.

Il dato più duro del lavoro è bibliometrico: esaminando 133 articoli recenti da ASPLOS, PACT, PLDI e CGO, nessuno degli articoli con risultati sperimentali considerava adeguatamente il bias di misura.

Le contromisure che propongono sono due: analisi causale per rilevarlo, e *randomizzazione del setup* per evitarlo.

Che cosa comporta per noi

Le nostre latenze sono state raccolte su una macchina, in una sessione, senza randomizzazione e senza controllo documentato su frequency scaling, pinning e isolamento delle CPU. La dispersione osservata — fino al 178% fra minimo e massimo nell'ultima esecuzione — è coerente con un ambiente rumoroso.

4.2 Un microbenchmark più veloce non è un sistema più veloce

Un'analisi recente specificamente su eBPF [12] sostiene che quando un'ottimizzazione rende un ciclo stretto di BPF_PROG_TEST_RUN più veloce del 18%, questa non è evidenza sufficiente per abilitarla: l'ottimizzazione può essere del tutto corretta e restare una cattiva scelta in produzione.

I meccanismi elencati sono concreti: un'ottimizzazione può accelerare un ciclo aritmetico ma aumentare la dimensione del codice fino a danneggiare la cache istruzioni in un programma più grande; può giovare a un backend JIT e non fare nulla su un altro; può ottimizzare un programma che pesa quasi nulla sul tempo totale dell'applicazione. E soprattutto: i benchmark isolati misurano singole operazioni, mentre i programmi reali combinano parsing, lookup su mappe, salti e riscrittura del pacchetto in un'unica catena, con caratteristiche che dipendono dall'interazione fra queste operazioni.

Che cosa comporta per noi

È una descrizione precisa della nostra situazione. La nostra Pipeline 1 ha 997 istruzioni statiche di cui ne esegue una frazione, e la tesi lo argomenta già bene. Ma il corollario vale anche al contrario: il vantaggio della hardcoded misurato in sandbox potrebbe ridursi sotto traffico reale, dove il costo del redirect e la pressione sulla cache sono gli stessi per tutte e tre.

4.3 Ambienti astratti non predicono ambienti reali

Il terzo risultato viene dal lato routing, ed è il più vicino al nostro scenario. Boltres et al. [9] studiano l'ottimizzazione del routing appresa e trovano che i modelli di rete "fluidi" — quelli astratti, che non modellano l'interazione a livello di pacchetto — non sono adeguati per adattamenti su scala di millisecondi. La conclusione sperimentale è netta: le strategie apprese addestrate in ambienti fluidi **non generalizzano** al setup più realistico a livello di pacchetto.

Osservano inoltre che gli ambienti di valutazione standardizzati nati per favorire la riproducibilità nella ricerca sul traffic engineering restano limitati proprio perché usano modelli di rete astratti.

Che cosa comporta per noi

Il nostro modello decide un reroute su scala di pacchetto. Valutarlo esclusivamente in un sandbox che non consegna pacchetti è la versione eBPF dello stesso problema.

Dove siamo

5.1 Quadro sinottico

La tabella seguente confronta, asse per asse, quello che la letteratura riporta con quello che il progetto produce oggi.

Metrica	Stato nel progetto	Giudizio
Accuratezza del modello (float)	assente	manca
Accuratezza quantizzata int8	assente	manca
Delta di quantizzazione	assente	manca
Equivalenza fra pipeline	riferimento intero in Python, confronto su classe scelta	solido , oltre la media
Architetture alternative	65-8-7, 65-4-4-4-7, multi-modello concorrente	solido
Latenza (microbenchmark)	minimo su 7-15 trial, p50, max, dispersione	buono ma campione esiguo
Latenza (end-to-end)	assente	manca
Throughput misurato	assente; il valore riportato è derivato da 1/latenza	da correggere
Throughput a perdita nulla	assente	manca
Sweep sulle dimensioni di frame	assente	manca
Istruzioni eBPF (xlated)	misurate	solido
Codice jited	misurato	solido
Lecture di mappa per pacchetto	misurate con build strumentata	solido
Memoria delle mappe	misurata dal kernel	solido (dopo correzione)
Costo di aggiornamento modello	misurato con syscall reali	solido, originale
Flessibilità architetturale	dimostrata su tre pipeline	solido, originale
Reroute su guasto	1 caso su 30 basta a passare	debole
Confronto con routing tradizionale	assente	manca
Controllo del rumore di misura	non documentato	manca

5.2 Dove siamo forti

Va detto con chiarezza, perché è un risultato reale. Il progetto è **sopra** la media della letteratura su tre punti.

Il primo è il criterio di correttezza. Confrontare la classe scelta dal programma eBPF contro un riferimento intero indipendente scritto in Python, che replica la stessa aritmetica, è più forte di quanto fanno molti lavori pubblicati, che si limitano a riportare un'accuratezza aggregata. Qui si verifica *l'equivalenza esatta*, decisione per decisione.

Il secondo è la trasparenza sulle metriche deterministiche. Distinguere la dimensione statica del programma dal percorso effettivamente eseguito, e spiegare perché dividere istruzioni per latenza darebbe un IPC impossibile, è esattamente il tipo di onestà che il lavoro di Mytkowicz lamenta essere assente.

Il terzo è l'asse 4. Il confronto costo-per-pacchetto contro costo-di-aggiornamento, misurato su tre punti di uno spazio di progetto reale, è un contributo che la letteratura tratta in modo meno sistematico.

5.3 Dove siamo scoperti

5.3.1 Non sappiamo se il modello è buono

È la lacuna principale. Il progetto dimostra in modo convincente che le tre pipeline calcolano *la stessa cosa*, ma non dimostra mai che quella cosa sia *giusta*. Un modello che sceglie sempre la classe 0 supererebbe tutti i test di equivalenza attuali.

Il test di reroute è l'unico che tocca il merito, ed è debole: passa se *almeno un caso* su trenta cambia l'uscita. Non verifica che il cambiamento sia quello corretto, perché non esiste una verità di riferimento contro cui confrontarlo.

5.3.2 Il throughput riportato non è un throughput

Il valore in Mpps nelle tabelle è calcolato come $1000/\text{latenza in ns}$. È una *proiezione* della latenza, non una misura: assume implicitamente serializzazione perfetta, nessuna perdita, nessun costo di I/O. In particolare, sotto BPF_PROG_TEST_RUN il redirect non avviene, quindi il costo maggiore del percorso reale è escluso. Chiamarlo throughput invita un confronto con i numeri della letteratura che non regge, perché quelli sono ottenuti a perdita nulla con traffico reale.

5.3.3 Due feature del modello sono inerti

Il progetto documenta già, correttamente, che `ingress_iface` non contribuisce: gli `ifindex` reali dei nodi Kathara non corrispondono né alla tabella della Pipeline 1 né all'intervallo accettato da Pipeline 2 e 3. A questa va aggiunta la feature `node`, che nel datapath è indicizzata da `model_id` anziché da un identificatore di nodo: con un solo modello registrato produce sempre lo stesso contributo su ogni nodo.

Sono 58 delle 65 componenti dell'input vector — 6 di `ingress_iface` e 52 di `node` — che nel laboratorio non portano informazione. Il modello decide di fatto su `link_state` e TTL. Questo va detto, perché cambia l'interpretazione di qualunque risultato di accuratezza.

5.3.4 Una sola macchina, una sola sessione

Nessuna randomizzazione del setup, nessun controllo documentato su governor della frequenza, pinning, isolamento delle CPU o iperthreading. Con dispersioni fino al 178% e un campione di 7 trial, i confronti di latenza fra pipeline vicine non sono difendibili.

Che cosa fare

Il piano è ordinato per rapporto fra valore e costo. I primi due punti sono quelli che cambiano la tesi; gli altri la rafforzano.

6.1 Priorità 1 — Misurare la qualità del modello

È la lacuna più grave ed è anche, paradossalmente, la più economica da colmare: non serve il laboratorio, serve solo Python, e i pezzi esistono già nel repository.

1. **Costruire un insieme di test con verità di riferimento.** Per la topologia Germany50, generare scenari di guasto (fino ai 10 guasti simultanei già dichiarati come limite) e calcolare per ciascuno il next-hop corretto verso la destinazione con un algoritmo classico di cammini minimi. Questa è la verità di riferimento che oggi manca.
2. **Riportare l'accuratezza su tre livelli:** il modello PyTorch in virgola mobile; il riferimento intero in Python; il programma eBPF. Il primo confronto misura il costo della quantizzazione a 8 bit; il secondo verifica che l'implementazione nel kernel non introduca deviazioni. Il secondo confronto il progetto lo fa già — va solo riportato come accuratezza anziché come test binario.
3. **Dichiarare la baseline.** L'accuratezza va confrontata con quella di una politica banale, ad esempio “scegli sempre il next-hop del cammino minimo senza guasti”. Un modello che non batte la politica banale non ha valore, e questo confronto è ciò che rende il numero interpretabile.

Criterio di accettazione

Una tabella con accuratezza float, accuratezza int8, delta di quantizzazione, e accuratezza della politica banale, su un insieme di test dichiarato e riproducibile.

6.2 Priorità 2 — Rendere onesto il throughput

Due opzioni, non alternative.

La via minima, a costo quasi nullo: **rinominare la metrica**. Non “Throughput (Mpps)” ma “Tasso di inferenza derivato dalla latenza (Mpps)”, con una nota che dichiara che sotto BPF_PROG_TEST_RUN il redirect non viene eseguito. Questo elimina il problema di onestà senza nuove misure.

La via completa, se il tempo lo permette: **una misura di throughput vera** su due macchine collegate, o anche su una sola con due interfacce e namespace separati, con un generatore che saturi il collegamento. Non serve TRex a 100 Gbps: bastano pktgen del kernel o trafgen, purché si riporti il throughput a perdita nulla e almeno tre dimensioni di frame. Anche a 1 Gbps il risultato sarebbe un dato reale, e la differenza fra le tre pipeline sotto carico è esattamente ciò che la letteratura si aspetta di vedere.

Criterio di accettazione

O la metrica è rinominata e annotata, oppure esiste una misura a perdita nulla con generatore dichiarato, durata dichiarata e almeno tre dimensioni di frame.

6.3 Priorità 3 — Chiudere il ciclo end-to-end

La tesi dichiara già che il traffico UDP sulla porta 9999 non viene consegnato oltre il singolo salto, e che per questo la verifica usa `BPF_PROG_TEST_RUN`. È una giustificazione onesta ma resta una debolezza: il meccanismo di fast-reroute non è mai stato mostrato funzionare in rete.

Serve almeno *una* dimostrazione end-to-end: un percorso di pochi salti, con le pipeline attaccate, un flusso da `h_src` a `h_dst`, un link fatto cadere a metà flusso, e la prova che il traffico cambia uscita senza intervento del control plane. Non deve essere una misura di prestazioni — basta che sia una prova di funzionamento. Vale più di dieci tabelle di microbenchmark, perché è l'unica evidenza diretta della tesi centrale del lavoro.

6.4 Priorità 4 — Irrobustire le misure di latenza

Interventi a basso costo, tutti documentabili in due righe di metodologia:

- portare i trial da 7 a almeno 30, riportando mediana e intervallo di confidenza oltre al minimo;
- fissare il governor della frequenza su performance e documentarlo;
- pinnare il processo su una CPU isolata e dichiarare quale;
- randomizzare l'ordine delle pipeline fra le ripetizioni, che è la contromisura proposta da Mytkowicz et al. [8];
- ripetere su almeno due sessioni separate da un riavvio.

La scelta del minimo come stimatore resta difendibile in ambiente rumoroso [10], ma va affiancata da una misura di dispersione — cosa che il progetto già fa — e da un campione adeguato, cosa che oggi non fa.

6.5 Priorità 5 — Riparare o dichiarare le feature inerti

Due strade, entrambe accettabili, ma una va scelta esplicitamente.

Ripararle: risolvere gli ifindex reali all'avvio e fornire a Pipeline 2 e 3 una mappa da ifindex a porta logica; seminare su ogni nodo il proprio identificatore e usarlo per la feature `node` anziché `model_id`.

Dichiararle: scrivere nella tesi che nel laboratorio attuale il modello decide su `link_state` e TTL, e che le altre due feature sono predisposte ma inerti. È una limitazione perfettamente accettabile in una tesi, purché sia dichiarata e non lasciata al lettore da scoprire.

La scelta peggiore è la terza: riportare l'accuratezza senza menzionarle.

In sintesi

Il progetto misura bene il *costo* del machine learning nel data plane, su tre punti di uno spazio di progetto, con metriche deterministiche verificate e un criterio di equivalenza più rigoroso della media. Questo è il contributo, ed è solido.

Quello che manca è il *valore*: nessuna misura dice se il modello svolga bene il compito, quanto costi la quantizzazione, né se il meccanismo funzioni in rete. La letteratura considera queste tre cose obbligatorie — non come contorno, ma come prima tabella.

La buona notizia è che la lacuna più grave è anche la più economica da colmare. Costruire un insieme di test con verità di riferimento e riportare tre livelli di accuratezza non richiede il laboratorio, non richiede nuovo hardware, e riusa il riferimento Python già scritto. È il singolo intervento con il rapporto valore/costo più alto dell'intero elenco.

Bibliografia

- [1] C. Zheng, X. Hong, D. Ding, S. Vargaftik, Y. Ben-Itzhak, N. Zilberman, *In-Network Machine Learning Using Programmable Network Devices: A Survey*, IEEE Communications Surveys & Tutorials, vol. 26, no. 2, 2024. DOI: 10.1109/COMST.2023.3344351.
- [2] R. Parizotto, B. L. Coelho, D. C. Nunes, I. Haque, A. Schaeffer-Filho, *Offloading Machine Learning to Programmable Data Planes: A Systematic Survey*, ACM Computing Surveys, vol. 56, no. 1, art. 18, 2023. DOI: 10.1145/3605153.
- [3] G. Siracusano, S. Galea, D. Sanvito, M. Malekzadeh, G. Antichi, P. Costa, H. Haddadi, R. Bifulco, *Re-architecting Traffic Analysis with Neural Network Interface Cards*, 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22), 2022. <https://www.usenix.org/conference/nsdi22/presentation/siracusano>
- [4] C. Zheng, Z. Xiong, T. T. Bui, S. Kaupmees, R. Bensoussane, A. Bernabeu, S. Vargaftik, Y. Ben-Itzhak, N. Zilberman, *Ilsy: Hybrid In-Network Classification Using Programmable Switches*, IEEE/ACM Transactions on Networking, vol. 32, no. 3, 2024. DOI: 10.1109/TNET.2024.3364757.
- [5] C. Zheng, M. Zang, X. Hong, L. Perreault, R. Bensoussane, S. Vargaftik, Y. Ben-Itzhak, N. Zilberman, *Planter: Rapid Prototyping of In-Network Machine Learning Inference*, ACM SIGCOMM Computer Communication Review, vol. 54, no. 1, pp. 2–21, 2024. DOI: 10.1145/3687230.3687232.
- [6] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller, *The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel*, 14th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '18), 2018. DOI: 10.1145/3281411.3281443.
- [7] S. Bradner, J. McQuaid, *Benchmarking Methodology for Network Interconnect Devices*, RFC 2544, IETF, 1999. <https://www.rfc-editor.org/rfc/rfc2544.html>
- [8] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney, *Producing Wrong Data Without Doing Anything Obviously Wrong!*, 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09), 2009. DOI: 10.1145/1508244.1508275.
- [9] A. Boltres, N. Freymuth, P. Jahnke, H. Karl, G. Neumann, *Learning Sub-Second Routing Optimization in Computer Networks requires Packet-Level Dynamics*, Transactions on Machine Learning Research (TMLR), 2024. arXiv:2410.10377.
- [10] J. Chen, J. Revels, *Robust benchmarking in noisy environments*, arXiv:1608.04295, 2016.
- [11] The Linux Kernel documentation, *Running BPF programs from userspace*. https://docs.kernel.org/bpf/bpf_prog_run.html
- [12] eunomia, *When Is an eBPF Optimization Result Strong Enough to Ship?* <https://eunomia.dev/research/ebpf-optimization-evidence-contract/>